Лабораторная работа №2
Оценка позы ArUco-маркеров и построение 3D-сцены

Введение

Дополненная реальность (Augmented Reality, AR) представляет собой класс систем, в которых виртуальные объекты интегрируются в изображение реальной сцены таким образом, чтобы сохранялась геометрическая согласованность между реальными и синтетическими объектами. Ключевой задачей любой AR-системы является оценка положения и ориентации камеры относительно наблюдаемой сцены, то есть задача оценки позы (pose estimation).

В общем случае определение позы камеры может выполняться по естественным особенностям изображения (feature-based методы), по инерциальным данным (различным датчикам) или по заранее известным объектам в сцене. Однако методы, основанные на естественных признаках, обладают рядом ограничений: чувствительность к текстуре, освещению, повторяющимся структурам, а также сложность обеспечения высокой точности в реальном времени.

Для повышения устойчивости и точности в системах дополненной реальности широко используются искусственные визуальные маркеры. Маркер представляет собой специально сконструированный объект с известной геометрией и кодированной структурой, позволяющей:

- однозначно идентифицировать объект в кадре;
- получить устойчивые 2D–3D соответствия;
- вычислить положение и ориентацию маркера относительно камеры;
- задать локальную систему координат сцены.

В отличие от произвольных текстур, маркеры проектируются таким образом, чтобы их детекция была устойчива к изменениям масштаба, поворота и перспективных искажений. Наличие жёстко заданной геометрии (как правило, квадратной формы) существенно упрощает решение задачи оценки позы, поскольку координаты характерных точек маркера в собственной системе координат известны заранее.

Использование маркеров в AR-системах позволяет перейти от абстрактной задачи реконструкции сцены к более строго формализованной задаче определения жёсткого преобразования между системой координат камеры и системой координат маркера. После определения этого преобразования становится возможным корректное размещение виртуальных объектов в реальном пространстве с сохранением перспективы, масштаба и ориентации.

Таким образом, маркер выступает в роли геометрического «якоря» сцены: он фиксирует положение виртуальной системы координат в физическом мире и обеспечивает воспроизводимость пространственной конфигурации. В рамках данной лабораторной работы маркеры используются как опорные элементы для построения общей системы координат стола и размещения виртуальной трёхмерной сцены.

Системы координат и жёсткие преобразования

Используемые системы координат

Рассмотрим следующие системы координат:

- мировая система $\mathcal{W}$;
- система камеры $\mathcal{C}$;
- система маркера $\mathcal{M}$.

Каждая система координат является декартовой трёхмерной системой с ортонормированным базисом.

Пусть $X_{\mathcal{W}} \in \mathbb{R}^3$ — координаты точки в мировой системе координат, $X_{\mathcal{C}} \in \mathbb{R}^3$ — координаты той же точки в системе камеры.

Связь между ними задаётся жёстким преобразованием.

Жёсткое преобразование в трёхмерном пространстве

Жёстким (евклидовым) преобразованием называется отображение, сохраняющее расстояния и углы. В $\mathbb{R}^3$ оно состоит из поворота и переноса.

Пусть:

$$R \in SO(3), \qquad t \in \mathbb{R}^3.$$

Тогда преобразование из системы $\mathcal{W}$ в систему $\mathcal{C}$ записывается как

$$X_{\mathcal{C}} = RX_{\mathcal{W}} + t, \tag{1}$$

где R — матрица поворота, t — вектор переноса.

Матрица поворота принадлежит специальной ортогональной группе:

$$R^T R = I, \tag{2}$$

$$\det R = 1. \tag{3}$$

Из ортогональности следует:

$$R^{-1} = R^T. \tag{4}$$

Это свойство существенно упрощает обращение жёсткого преобразования.

Однородное представление

Для удобства композиции преобразований используется однородная форма записи.

Точка в однородных координатах:

$$\tilde{X} = \begin{pmatrix} X \\ 1 \end{pmatrix}. \tag{5}$$

Жёсткое преобразование записывается как матрица 4×4:

$$T = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix}. \tag{6}$$

Тогда преобразование принимает вид:

$$\tilde{X}_{\mathcal{C}} = T \tilde{X}_{\mathcal{W}}. \tag{7}$$

Поскольку $R^{-1} = R^T$, обратное преобразование вычисляется аналитически:

$$T^{-1} = \begin{pmatrix} R^T & -R^T t \\ 0 & 1 \end{pmatrix}. \tag{8}$$

Это выражение играет важную роль при переходе между системами координат маркера, камеры и стола.

Композиция преобразований

Пусть заданы два преобразования:

$$T_1 = \begin{pmatrix} R_1 & t_1 \\ 0 & 1 \end{pmatrix}, \qquad T_2 = \begin{pmatrix} R_2 & t_2 \\ 0 & 1 \end{pmatrix}. \tag{9}$$

Их композиция определяется матричным произведением:

$$T_{12} = T_2 T_1. \tag{10}$$

В развернутом виде:

$$T_{12} = \begin{pmatrix} R_2 R_1 & R_2 t_1 + t_2 \\ 0 & 1 \end{pmatrix}. \tag{11}$$

Из этого выражения видно, что:

- повороты перемножаются,
- перенос второго преобразования добавляется после поворота первого.

Цепочки преобразований

Если точка задана в системе маркера $\mathcal{M}$, а требуется получить её координаты в системе камеры $\mathcal{C}$ через мировую систему $\mathcal{W}$, используется композиция:

$$T_{\mathcal{CM}} = T_{\mathcal{CW}} T_{\mathcal{WM}}. \tag{12}$$

Тогда:

$$\tilde{X}_{\mathcal{C}} = T_{\mathcal{CM}} \tilde{X}_{\mathcal{M}}. \tag{13}$$

Именно такие цепочки преобразований координат лежат в основе размещения виртуальных объектов в сцене дополненной реальности.

Геометрический смысл

Жёсткое преобразование полностью определяет:

- положение начала координат одной системы относительно другой;
- ориентацию базисных векторов;
- относительное расположение всех точек.

В задачах дополненной реальности оценка позы маркера фактически означает вычисление матрицы $T_{\mathcal{CM}}$. После её получения любая точка виртуального объекта, заданная в системе маркера или стола, может быть корректно перенесена в систему камеры и затем спроецирована на изображение.

ArUco-маркеры и процесс их детекции

ArUco-маркер, рис. 1, представляет собой квадратный бинарный шаблон с кодированной внутренней структурой и чётко выделенной границей. Геометрия маркера (квадрат) и дискретная структура кода позволяют получать устойчивые 2D–3D соответствия, необходимые для решения задачи оценки позы.

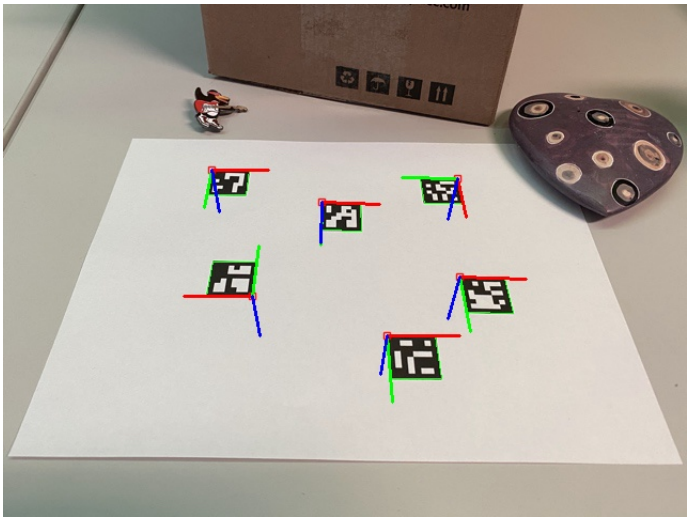

Рис. 1: ArUco маркеры.

Каждый маркер принадлежит некоторому словарю. Словарь определяет:

- размер внутренней бинарной сетки (4×4, 5×5, 6×6 и т.д.);
- количество уникальных идентификаторов;
- минимальное кодовое расстояние между маркерами.

Чем больше размер сетки, тем выше потенциальная устойчивость к ошибкам и больше количество уникальных ID, однако усложняется детекция на больших расстояниях.

Процесс детекции маркера состоит из последовательности этапов:

1. Предварительная обработка и бинаризация изображения;
2. Поиск квадратных контуров;
3. Проверка геометрических ограничений;

4. Перспективное выравнивание (warp);
5. Декодирование бинарного шаблона;
6. Проверка словаря и коррекция ошибок.

Бинаризация изображения

Цель первого этапа — выделить потенциальные контуры маркеров.

Исходное изображение переводится в градации серого:

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY))
```

Далее применяется адаптивная или глобальная пороговая обработка:

```
thresh = cv2.adaptiveThreshold(...)
```

Адаптивная бинаризация предпочтительна, поскольку она компенсирует неравномерное освещение, повышает устойчивость к теням, облегчает выделение границ маркера.

Результатом является бинарное изображение, на котором потенциальные маркеры выделяются как контрастные квадраты.

Поиск контуров и геометрическая фильтрация

На бинарном изображении выполняется поиск контуров, рис. 2.

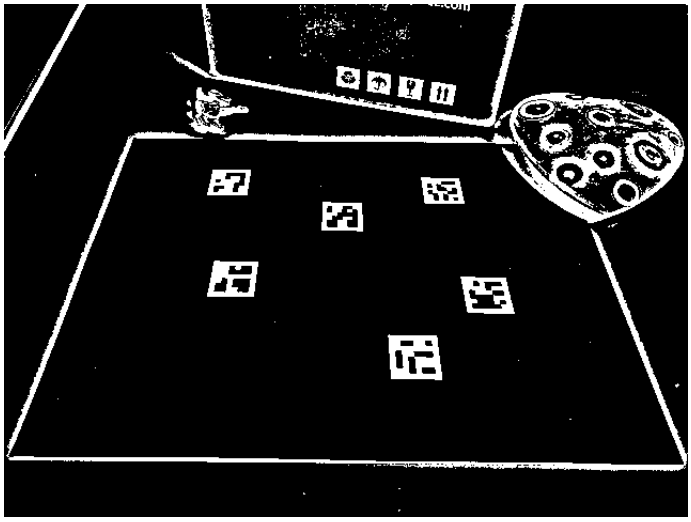

Рис. 2: Контуры.

Для каждого контура проверяется замкнутость, площадь (фильтрация слишком маленьких или больших областей), приближение к четырёхугольнику.

После выполнения аппроксимации, например методом Дугласа–Пекера, как кандидаты принимаются те контуры, что:

- содержат ровно 4 вершины;
- являются выпуклым;
- имеют разумные пропорции сторон.
- имеют корректную ориентация вершин (порядок обхода);
- не содержат самопересечений;
- имеют допустимое перспективное искажение.

Цель — оставить только те четырёхугольники, которые могут быть перспективной проекцией квадрата.

Перспективное выравнивание

Для каждого кандидата вычисляется гомография, переводящая найденный четырёхугольник в нормализованный квадрат фиксированного размера.

Пусть заданы четыре точки изображения x_i и соответствующие точки канонического квадрата $X_{\mathcal{M},i}$. Тогда существует матрица гомографии H, такая что:

$$x_i \sim H X_{\mathcal{M},i}. \tag{14}$$

После вычисления гомографии выполняется перспективное преобразование (warp):

```
marker = cv2.warpPerspective(gray, H, (N, N))
```

Результат — изображение маркера в нормализованной системе координат без перспективных искажений.

Бинарное декодирование

Нормализованное изображение разбивается на сетку $n \times n$, соответствующую структуре словаря.

Для каждой ячейки вычисляется средняя интенсивность, и принимается решение:

$$b_{ij} = \begin{cases} 1, & \text{если интенсивность ниже порога,} \\ 0, & \text{иначе.} \end{cases}$$

Полученная бинарная матрица интерпретируется как код маркера.

Важно учитывать возможные повороты маркера кратные $\frac{\pi}{2}$. Для каждого поворота формируется соответствующий код и сравнивается со словарём.

Проверка словаря и коррекция ошибок

Каждый словарь характеризуется минимальным расстоянием Хэмминга между кодами.

Расстояние Хэмминга определяется как:

$$d_H(a, b) = \sum_i |a_i - b_i|. \tag{15}$$

Если расстояние до ближайшего допустимого кода меньше допустимого порога, маркер считается распознанным. Наличие такого кодового расстояния позволяет корректировать отдельные битовые ошибки и снижать вероятность ложных срабатываний.

Формирование результата детекции

После успешного декодирования формируются:

- координаты четырёх углов маркера в изображении;
- идентификатор маркера (ID);
- порядок вершин.

В OpenCV итоговый вызов имеет вид:

```
corners, ids, rejected = cv2.aruco.detectMarkers(frame, dictionary)
```

Полученные координаты углов служат входными данными для решения задачи PnP и последующей оценки позы.

При этом важно помнить, что данный процесс детекции чувствителен к сильным перспективным и нелинейным искажениям, малому размеру и частичной заслонённости маркера. При уменьшении видимого размера маркера уменьшается количество пикселей, соответствующих одной ячейке сетки, что приводит к росту ошибок бинаризации и декодирования.

Таким образом, ArUco-маркеры представляют собой специально сконструированный компромисс между информационной ёмкостью, устойчивостью к шуму и вычислительной эффективностью детекции.

Оценка позы и задача Perspective-n-Point

Оценка позы (pose estimation) — ключевой этап построения системы дополненной реальности. Под оценкой позы понимается определение положения и ориентации объекта (или камеры) относительно другой системы координат.

В контексте данной лабораторной работы требуется определить жёсткое преобразование между системой координат маркера и системой координат камеры. Математически эта задача сводится к классической задаче **Perspective-n-Point (PnP)**.

Постановка задачи PnP

Пусть заданы:

- набор трёхмерных точек $X_{\mathcal{M},i} \in \mathbb{R}^3$ в системе координат объекта;
- соответствующие им точки изображения $x_i \in \mathbb{R}^2$;
- матрица внутренних параметров камеры K.

Требуется определить матрицу поворота R и вектор переноса t, такие что выполняется соотношение проекции:

$$x_i \sim K(RX_{\mathcal{M},i} + t) \tag{16}$$

Знак "$\sim$" означает равенство с точностью до масштаба в проективном пространстве.

Иначе говоря, необходимо найти такое жёсткое преобразование, при котором проекции трёхмерных точек максимально согласуются с наблюдаемыми точками изображения.

Для решения задачи необходимо минимум четыре некомпланарные точки.

В случае ArUco-маркера используются четыре угла квадрата, координаты которых известны в локальной системе координат маркера. Это делает маркер естественным объектом для постановки задачи PnP.

Ошибка репроекции

Поскольку измерения содержат шум, задача решается как задача оптимизации. Определяется ошибка репроекции:

$$e_i = x_i - \hat{x}_i(R, t), \tag{17}$$

где $\hat{x}_i(R, t)$ — проекция точки X_i при текущих параметрах.

Суммарная квадратичная ошибка:

$$E(R,t) = \sum_{i=1}^{n} \|e_i\|^2. \tag{18}$$

Задача PnP сводится к минимизации функционала (18) по переменным R и t.

Параметризация поворота

Матрица поворота имеет 9 элементов, но лишь 3 степени свободы. В численных алгоритмах поворот удобно задавать вектором Родригеса $r \in \mathbb{R}^3$.

Связь между вектором Родригеса и матрицей поворота определяется экспоненциальным отображением:

$$R = \exp([\omega]_\times), \tag{19}$$

где $[\omega]_\times$ — кососимметричная матрица, соответствующая вектору угловой скорости. В OpenCV данный переход выполняется следующим образом:

```
R, _ = cv2.Rodrigues(rvec)
```

Таким образом, оптимизация ведётся по параметрам $rvec$ и $tvec$.

Алгоритмы решения PnP

Существует несколько классов алгоритмов:

- аналитические (P3P);
- линейные приближения (EPnP);
- итерационные методы (Levenberg–Marquardt).

Итерационные методы минимизируют ошибки репроекции напрямую и обеспечивают высокую точность при хорошем начальном приближении.

В OpenCV используется универсальный интерфейс:

```
retval, rvec, tvec = cv2.solvePnP(objPts, imgPts, K, dist)
```

где:

- `objPts` — массив 3D-точек;
- `imgPts` — соответствующие 2D-точки;
- `K` — матрица внутренних параметров;
- `dist` — коэффициенты дисторсии.

Геометрическая интерпретация результата

Результат решения PnP:

- $rvec$ — ориентация объекта относительно камеры;
- $tvec$ — положение начала системы объекта в системе камеры.

Вектор переноса имеет прямой геометрический смысл:

$$tvec = \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix}, \tag{20}$$

где t_z соответствует расстоянию до объекта вдоль оптической оси камеры.

Матрица поворота определяет ориентацию локальных осей объекта в системе камеры.

Итоговое жёсткое преобразование:

$$T_{\mathcal{CM}} = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \tag{21}$$

Однако важно отметить, что задача PnP может иметь несколько решений (в случае малого числа точек) и является чувствительной к шуму, особенно возрастающему при малых углах наблюдения.

После получения R и t любая точка виртуального объекта, заданная в системе координат маркера, может быть преобразована в систему камеры, а затем спроецирована на изображение. Таким образом, задача PnP является центральным вычислительным механизмом, обеспечивающим геометрическую согласованность виртуальной и реальной сцены.

Проекция 3D-объектов на изображение

После оценки позы объекта (маркера или стола) в системе координат камеры следующим этапом является отображение виртуальных трёхмерных объектов на изображение. Геометрически эта операция состоит из двух последовательных преобразований:

1. переход из системы координат объекта в систему координат камеры;
2. перспективная проекция точки из пространства $\mathbb{R}^3$ на плоскость изображения.

Переход из системы объекта в систему камеры

Пусть точка виртуального объекта задана в системе координат $\mathcal{M}$:

$$X_{\mathcal{M}} = \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}.$$

Результатом решения задачи PnP является матрица поворота R и вектор переноса t, определяющие преобразование из системы объекта в систему камеры:

$$X_{\mathcal{C}} = RX_{\mathcal{M}} + t. \tag{22}$$

Здесь

$$X_{\mathcal{C}} = \begin{pmatrix} X_c \\ Y_c \\ Z_c \end{pmatrix}$$

— координаты точки в системе камеры.

Перспективная проекция

В модели pinhole-камеры точка проецируется на плоскость изображения по закону перспективы:

$$x' = \frac{X_c}{Z_c}, \qquad y' = \frac{Y_c}{Z_c}. \tag{23}$$

Это нормализованные координаты в плоскости, расположенной на расстоянии $f = 1$ от центра проекции.

С учётом реальных параметров камеры используется матрица внутренних параметров:

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \tag{24}$$

Тогда координаты пикселя вычисляются как

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}. \tag{25}$$

Подставляя (23), получаем итоговую формулу проекции:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} \frac{X_c}{Z_c} \\ \frac{Y_c}{Z_c} \\ 1 \end{pmatrix}. \tag{26}$$

В развернутом виде:

$$u = f_x \frac{X_c}{Z_c} + c_x, \tag{27}$$

$$v = f_y \frac{Y_c}{Z_c} + c_y. \tag{28}$$

Таким образом, размер объекта на изображении обратно пропорционален его глубине Z_c.

Полная формула проекции

Объединяя (22) и (26), получаем компактную запись:

$$x \sim K(RX_{\mathcal{M}} + t). \tag{29}$$

Именно это выражение лежит в основе функции `projectPoints`.

```
imgPts, _ = cv2.projectPoints(points3D, rvec, tvec, K, dist)
```

Функция выполняет:

- преобразование координат через R и t;
- деление на глубину Z_c;
- применение матрицы внутренних параметров;
- учёт дисторсии (если задана).

Построение куба в локальной системе координат

Пусть куб имеет ребро длины L и его нижняя грань лежит в плоскости $Z = 0$ системы координат объекта.

Вершины куба:

$$\begin{aligned}&(0,0,0),(L,0,0),(L,L,0),(0,L,0),\\&(0,0,L),(L,0,L),(L,L,L),(0,L,L)\end{aligned} \tag{30}$$

Каждая вершина рассматривается как точка $X_{\mathcal{M}}$ и подставляется в формулу (29).

Алгоритм переноса куба на изображение

Процесс отображения куба состоит из последовательных шагов:

1. Задать массив из 8 вершин куба в системе координат объекта.
2. Преобразовать каждую вершину в систему камеры.
3. Выполнить перспективную проекцию.
4. Соединить соответствующие пары точек рёбрами.

После получения двумерных координат:

```
cv2.line(frame, pt1, pt2, thickness)
```

соединяются рёбра: четыре нижних ребра, четыре верхних, четыре вертикальных.

Геометрический смысл

Если преобразование R, t оценено корректно, то:

- куб будет жёстко «привязан» к маркеру;
- при движении камеры перспектива будет изменяться естественным образом;
- размер куба будет уменьшаться при увеличении расстояния;
- параллельные рёбра будут сходиться в точках схода.

Таким образом, перенос куба на изображение является прямым следствием последовательного применения жёсткого преобразования и перспективной проекции. Именно эта цепочка операций обеспечивает пространственную согласованность виртуального и реального мира в системе дополненной реальности.

Задание

Камера наблюдает сцену в реальном времени. Параметры калибровки определить самостоятельно.

Необходимо разработать систему дополненной реальности, которая:

- детектирует маркеры в реальном времени;
- оценивает их позу;
- формирует систему координат стола;
- строит виртуальную пирамиду из кубиков;
- корректно размещает пирамиду на плоскости стола;
- отображает полученное изображение.

Архитектура решения полностью определяется студентом.